LECTURE: 2 BASIC OF C++ (PART - 3)

Mondira Chakraborty
Lecturer, CSE Dept.
Notre Dame University Bangladesh

Content

1. What is Pointer?
2. Why use Pointers?
3. Pointer Example
4. Notes
5. Swap two numbers using Pointers
6. Null Pointer
7. Null Pointer Example
8. Why use Null Pointers?
9. Double Pointer
10. Double Pointer Example

What is Pointer?

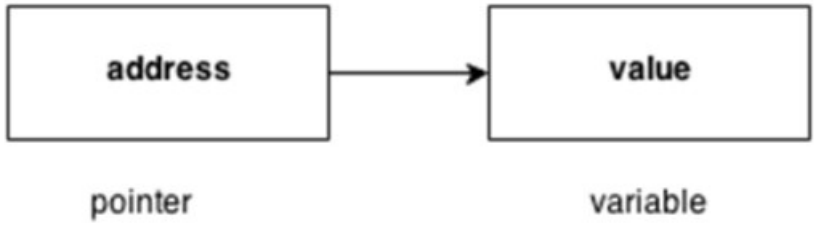
- A pointer is a variable that stores a memory address.
- Refers to some address in the program's memory space.
- Used to store the addresses of other variables or memory items.
- Very useful for another type of parameter passing, usually referred to as **Pass By Address**.
- Essential for dynamic memory allocation.

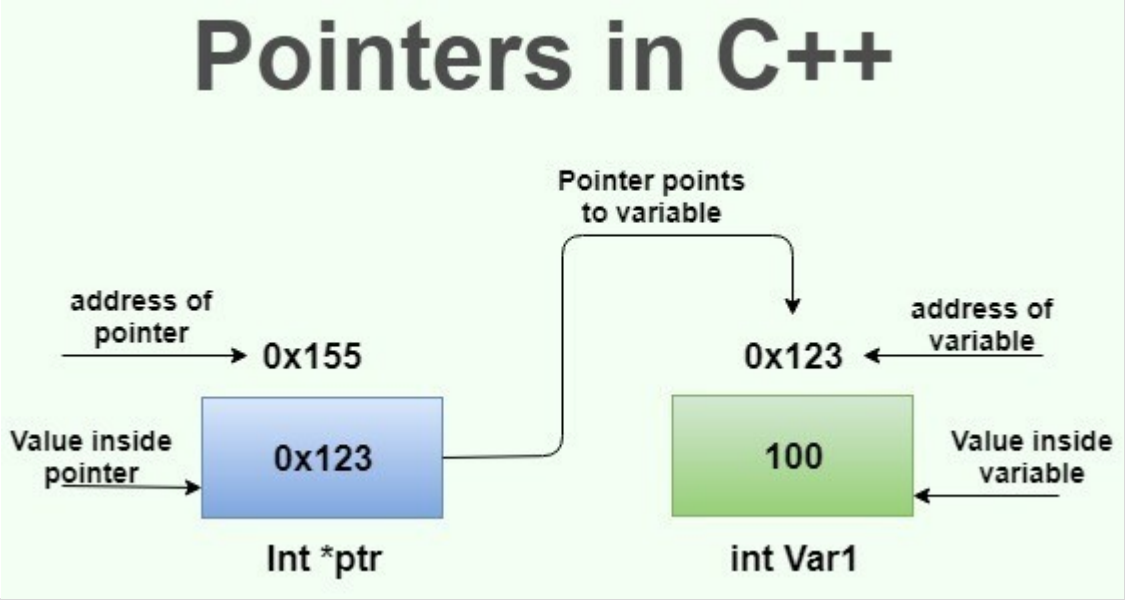
Why use Pointer?

- Short Ans: Don’t! ⏪ If it’s not necessary then don’t use it.
- Execution time with pointers is faster because data are manipulated with the address, that is, direct access to memory location.
- The pointer assigns and releases the memory as well. Hence it can be said the Memory of pointers is dynamically allocated.
- Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.
- An array, of any type, can be accessed with the help of pointers, without considering its subscript range.
- Pointers are used for file handling.

Pointer Example 1

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  int Var1 = 100;

  cout <<"Var1 = " <<Var1 << "\n";
  cout <<"&Var1 = " << &Var1 << "\n\n";

  int *ptr = &Var1;

  cout << "ptr = " << ptr << "\n";
  cout << "*ptr = " << *ptr << "\n";

  return 0;
}
```





Pointer Example 2

```
#include <iostream>
#include <string>
using namespace std;

int main() {
  string A = "NDUB Programming Club";  // A string variable
  string *ptr = &A;  // A pointer variable that stores the address of A

  // Output the value of A
  cout << A << "\n";

  // Output the memory address of A
  cout << &A << "\n";

  // Output the memory address of A with the pointer
  cout << ptr << "\n";

  // Output the data of A with the pointer
  cout << *ptr << "\n";
  return 0;
}
```

Notes

- Note that the * sign can be confusing here, as it does two different things in our code

- When used in declaration (string* ptr), it creates a **pointer variable**.

- When not used in declaration, it act as a **dereference operator**.

- The **dereferencing operator (*)** is a unary operator that returns the value present at the specified address.
- The **referencing operator (&)** is a unary operator that returns the address of its operand variable.

Swap two numbers using Pointers

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
int a=20,b=10,*p1=&a,*p2=&b;

cout<<"Before swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
*p1=*p1+*p2;
*p2=*p1-*p2;
*p1=*p1-*p2;


cout<<"After swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
   return 0;
}
```

Null Pointer

- There is a special pointer whose value is 0. It is called the *null pointer*
- You **can** assign 0 into a pointer
- The null pointer is **never** a valid target, however. If you try to dereference the null pointer, you WILL get a segmentation fault
- So why use it? The null pointer is typically used as a placeholder to initialize pointers until you are ready to use them (i.e. with valid targets), so that their values are **known**

Null Pointer Example

```
#include <iostream>
using namespace std;

int main()
{
    int* ptr = NULL; //you can use both NULL and nullptr to create null pointer
    // Checking if the pointer is null before dereferencing
    if (ptr == nullptr) {
        cout << "Pointer is currently null." << endl;
    }
    else {
        cout << "Pointer is not null." << endl;
    }

    // *ptr = 10; (to avoid runtime error)
    // Assigning a valid memory address to the pointer
    int value = 5;
    ptr = &value;

    // Checking if the pointer is null after assigning a
    // valid address
    if (ptr == nullptr) {
        cout << "Pointer is currently null." << endl;
    }
    else {
        cout << "Pointer is not null." << endl;
        cout << "Value at the memory location pointed to "
                "by the pointer: " << *ptr << endl;
    }

    return 0;
}
```

Applications of Null Pointers

- **Initialization:** It is a good practice to Initialize pointers to a null value as it helps avoid undefined behavior by explicitly indicating they are not pointing to valid memory locations.

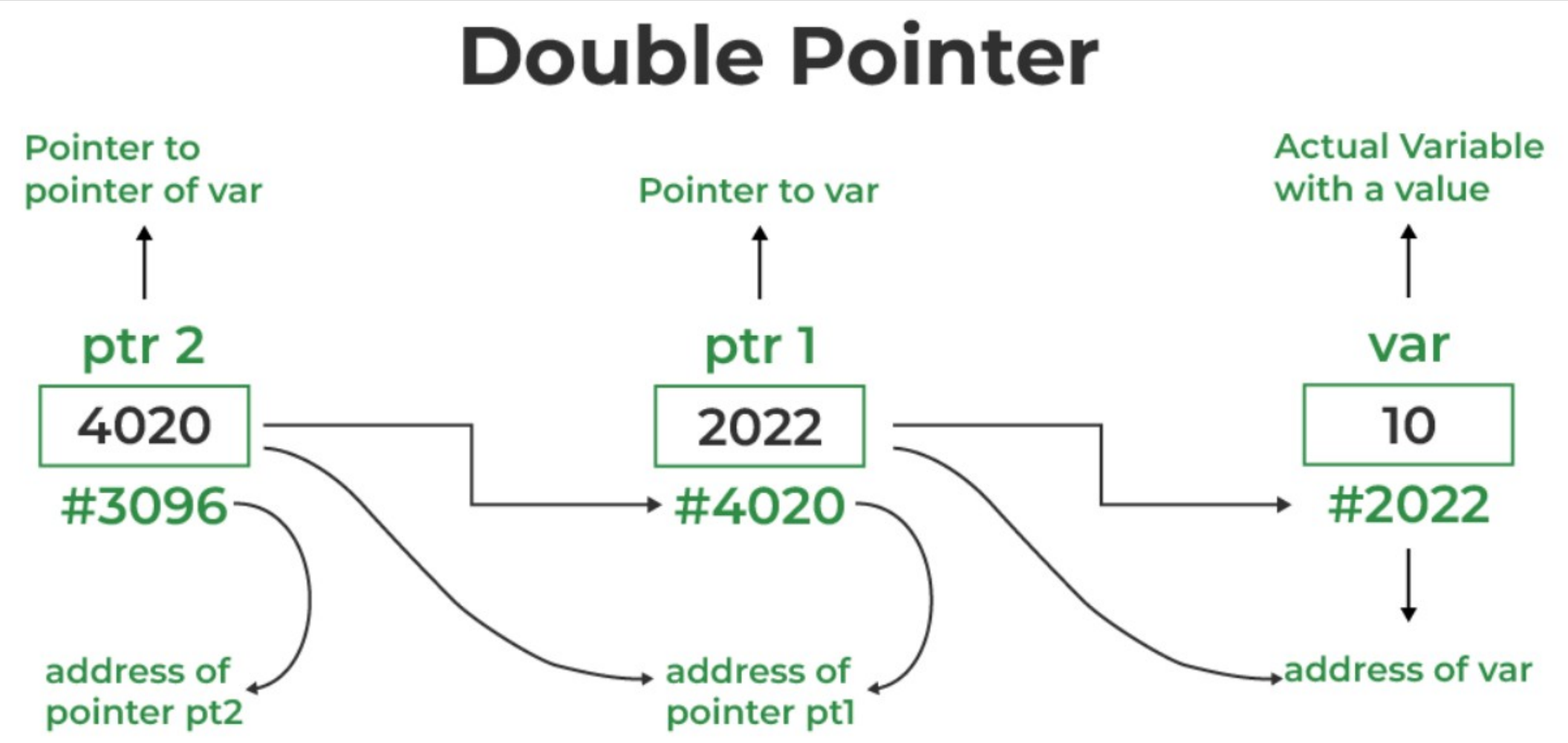
- **Default Values:** Null pointers act as default or initial values for pointers when no valid address is assigned to the pointers.

- **Error Handling:** They are useful in error conditions or to signify the absence of data that enables better handling of exceptional cases.

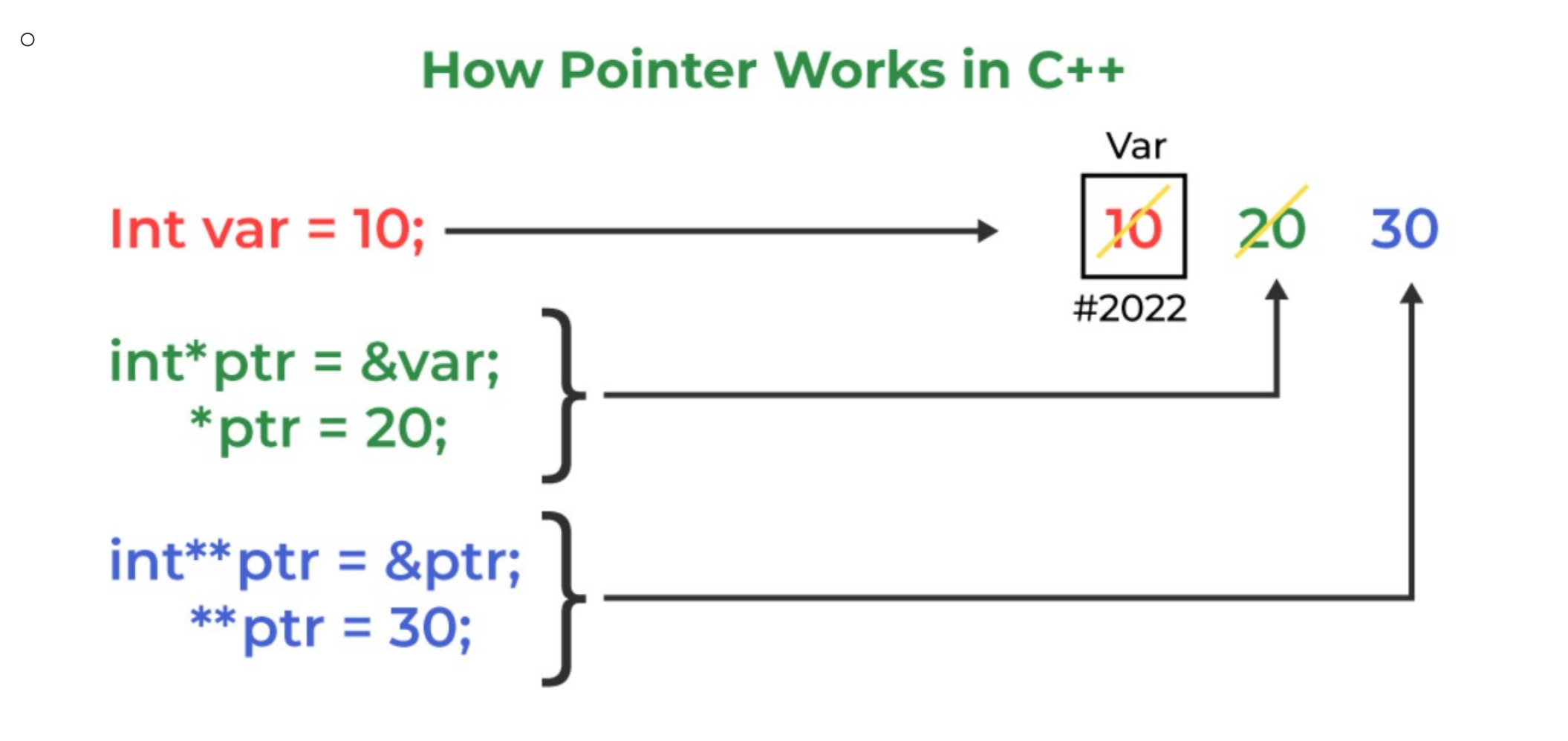
Double Pointer

- Now, we already know that a pointer stores the memory address of other variables.
- So, when we define a pointer to a pointer, the first pointer is used to store the address of the variables, and the second pointer stores the address of the first pointer.
- For this very reason, this is known as a **Double Pointer** or **Pointer to Pointer**

Double Pointer



Double Pointer



Double Pointer Example

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
  int variable = 169;

  // Pointer to store the address
  // of variable
  int* pointer1;
  // double pointer to store the
  // address of pointer1
  int** pointer2;
  // Storing address of variable
  // in pointer1
  pointer1 = &variable;
  // Storing address of pointer1
  // in pointer2
  pointer2 = &pointer1;
  cout << "Value of variable :- " <<
          variable << "\n";
  cout << "Value of variable using single pointer :- " <<
          *pointer1 << "\n";
  cout << "Value of variable using double pointer :- " <<
          **pointer2 << "\n";
  return 0;
}
```

Array using Pointers

```
#include<bits/stdc++.h>

using namespace std;

int main()
{
  int arr[5] = { 1, 2, 3, 4, 5 };
  int *ptr = arr;

  cout << ptr << endl;
  // Get the value of the second element in ptr
  cout << *(ptr + 1) << endl;
  // Get the value of the third element in ptr
  cout << *(ptr + 2) << endl;

  //Using for loop to access array
  for (int i = 0; i < 5; i++)
  {
      cout << *(ptr + i) << endl;
  }
  return 0;
}
```

References:

1. https://www.cs.fsu.edu/~myers/c++/notes/pointers1.html
2. https://simplesnippets.tech/cpp-pointers-concept-with-example/
3. https://www.w3schools.com/cpp/cpp_pointers.asp
4. https://www.javatpoint.com/cpp-pointers
5. https://www.geeksforgeeks.org/cpp-pointer-to-pointer-double-pointer/
6. https://www.geeksforgeeks.org/what-is-dynamic-memory-allocation/